
Conditional PixelCNN++ for Generative Modeling and Classification

Aaran Poon 36228203
UBC CPEN455

Abstract

This work explores the generative and classification capabilities of PixelCNN++ through label conditioning and fusion strategies integrated into the model’s architecture. We present comprehensive experiments, including ablation studies, quantitative evaluations via Bits Per Dimension (BPD) and classification accuracy, and qualitative image analysis.

1 Introduction

$$p_{\theta}(x) = \prod_{i=1}^n p_{\theta}(x_i | x_1, x_2, \dots, x_{i-1}) = \prod_{i=1}^n p_{\theta}(x_i | x_{<i})$$

Figure 1: Unconditional PixelCNN architecture

We begin with an unconditional PixelCNN++ architecture (Figure 1) and adapt it for conditional generation and classification by incorporating class information into the network.

2 Model Architecture

2.1 Overview

Our architecture extends the original PixelCNN++ with a U-Net-style encoder-decoder framework and added conditioning paths. The network contains skip connections for multi-scale feature propagation, allowing both fine- and coarse-level spatial information to influence pixel prediction.

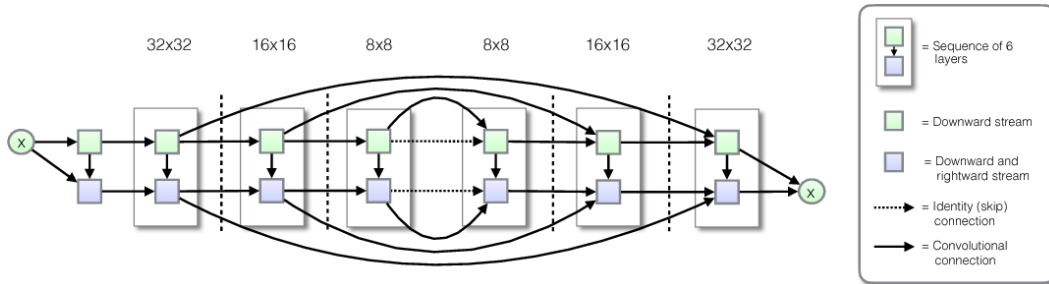


Figure 2: Layer diagram of Conditional PixelCNN++

2.2 Masked Convolution and Autoregressive Property

To ensure autoregressive behavior, we employ masked convolutions. Let $x_{i,j}$ denote the pixel at location (i, j) . We model the distribution $p(x)$ as a product of conditional distributions:

$$p(x) = \prod_{i,j} p(x_{i,j} | x_{<i,j}) \quad (1)$$

where $x_{<i,j}$ denotes all pixels preceding (i, j) in raster scan order. This is enforced using down-shifted and down-right-shifted convolutions.

2.3 Class Conditioning via Fusion Strategies

We condition the model on class labels y by embedding them using:

$$e_y = \text{Embedding}(y) \in \mathbb{R}^d \quad (2)$$

We use multiple fusion mechanisms:

Addition (Early + Middle):

The embedding e_y is reshaped to $[B, d, 1, 1]$ and added to feature maps:

$$F' = F + e_y \quad (3)$$

FiLM Modulation:

We adopt FiLM layers (2), which apply feature-wise affine transformations:

$$\text{FiLM}(F, e_y) = \gamma(e_y) \cdot F + \beta(e_y) \quad (4)$$

where γ and β are two-layer MLPs with hidden dimension 128. This allows flexible control over activations.

2.4 Mixture of Logistics for Pixel Modeling

Output pixel intensities are modeled via a mixture of discretized logistics (MDL):

$$p(x) = \sum_{k=1}^K \pi_k \cdot \text{Logistic}(x | \mu_k, s_k) \quad (5)$$

where π_k is the mixture weight, μ_k is the mean, and s_k is the scale for the k -th component. This formulation captures multimodality and improves likelihood estimates.

One major advantage of using a mixture of logistics is its ability to enable differentiable, stable, and flexible pixel sampling. In the sampling function, pixels are generated autoregressively using the model’s output distribution over mixture parameters. The discrete logistic mixture allows precise control over the output space and enables proper sampling of pixel intensities with spatial and inter-channel dependencies (via learned coefficients). For RGB images, each channel’s value is conditioned on the previously generated ones, handled cleanly by the mixture design.

This philosophy is also adopted in modern models such as DeepSeek-v2/v3, where mixture distributions (e.g., Gaussian or logistic mixtures) are used to better capture the distribution of continuous or quantized data. Compared to softmax over pixel values (as in older PixelCNNs), this yields higher-fidelity samples and better likelihood modeling while being more memory-efficient.

The final output layer projects to 10×3 channels (for RGB), encoding mixture logits, means, log scales, and inter-channel coefficients.

3 Experiments

3.1 Training Setup and Results

We trained and evaluated 7 model variants using different conditioning strategies and hyperparameters.

Early+Middle Addition:

Adding class embeddings at both early and mid-layers resulted in modest performance. Using 3 and 5 ResNet blocks (140 filters), classification accuracy plateaued around 49% (400 epochs), with an FID of 22. Increasing depth to 5 ResNet blocks led to overfitting, worsening FID and accuracy.

FiLM Conditioning:

FiLM layers were inserted into each gated residual block. Using 3 ResNet blocks and 120 filters, accuracy improved to 76%, albeit with a slightly worse FID (25).

FiLM + Nonlinear Hidden Layer:

Inspired by Squeeze-and-Excitation Networks (3), we added a hidden layer in the FiLM module to enhance expressiveness. With 3 ResNet blocks and 80 filters, this configuration achieved 83% accuracy and a lower FID (23.5). Increasing filters to 120 slightly degraded accuracy (81%) without improving FID.

3.2 Stochastic Weight Averaging (SWA) Optimization

To further improve generalization, we applied SWA beginning at epoch 350. This technique averages weights across checkpoints near the end of training (4). Final accuracy improved to 85%, and BPD dropped from 2.85 to 2.82.

Table 1: Comparison of model configurations: FID vs. classification accuracy

Model Configuration	FID	Classification Accuracy
450 epoch SWA + FiLM + hidden layer (80 filters)	23.58	85%
350 epoch FiLM + hidden layer (80 filters)	24.00	83.4%
300 epoch FiLM + hidden layer (120 filters)	25.62	81.1%
300 epoch FiLM + hidden + SWA	24.36	82.3%
350 epoch FiLM (no hidden, 120 filters)	24.80	70%
400 epoch Early+Middle Add, 3 ResNet, 140 filters	22.00	49%
150 epoch Early+Middle Add, 5 ResNet, 140 filters	27.50	30%

3.3 Sample Quality Analysis

Sample quality was assessed using both FID and visual inspection. FiLM with hidden layers produced visibly cleaner samples compared to additive fusion.

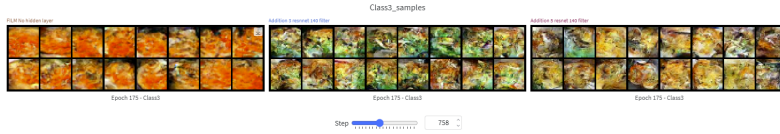


Figure 3: Comparison: FiLM vs. addition conditioning

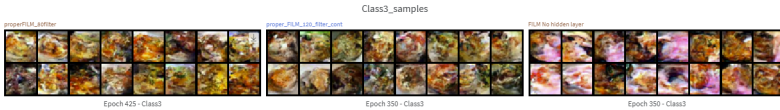


Figure 4: FiLM variants: with and without hidden layers

4 Conclusion

Our final model integrates multiple improvements from prior work to enhance performance:

- FiLM conditioning (2) enables feature modulation, allowing label embeddings to guide generative processes.
- Hidden layers in FiLM modules adopt ideas from Squeeze-and-Excitation Networks (3), improving adaptive feature scaling.
- Stochastic Weight Averaging (4) stabilizes convergence and enhances generalization.

These collectively resulted in 85% classification accuracy and 23.58 FID—substantial improvements over baseline PixelCNN++ models. Although classification lags behind task-specific CNNs such as ResNet-18 (~95.5%), our model is capable of full generative modeling with uncertainty estimation and label-conditional sampling.

While the epoch count between the models differs slightly, they were selected when the epoch iteration gave the better validation accuracy and FID score. Part of the limitation is that the best performing one was also the one with the highest epoch count therefore there might be more slight performance increase if other models were also trained for longer. However, they all hit a plateau in validation accuracy as well as validation BPD so increasing training time does not necessarily mean it will perform better than the current iterations.

5 Future Work

- **Self-Attention:** Integrate PixelSNAIL-style self-attention blocks to better capture long-range spatial dependencies (6).
- **Fusion Exploration:** Systematic comparison of FiLM, addition, and concatenation at various layers to find optimal conditioning depth.
- **Advanced Optimization:** Try AdamW and learning rate schedules (e.g., cosine decay) in combination with SWA.
- **Regularization:** Apply augmentations like CutMix to reduce overfitting in deeper variants.
- **Extended Training:** Run for longer durations with periodic SWA to further stabilize convergence.

Appendix

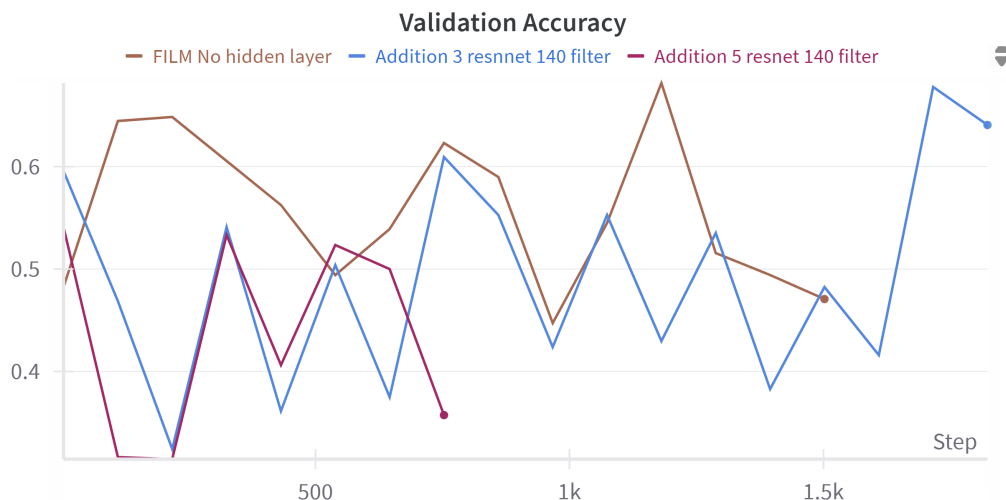


Figure 5: Validation accuracy: FiLM vs. addition conditioning

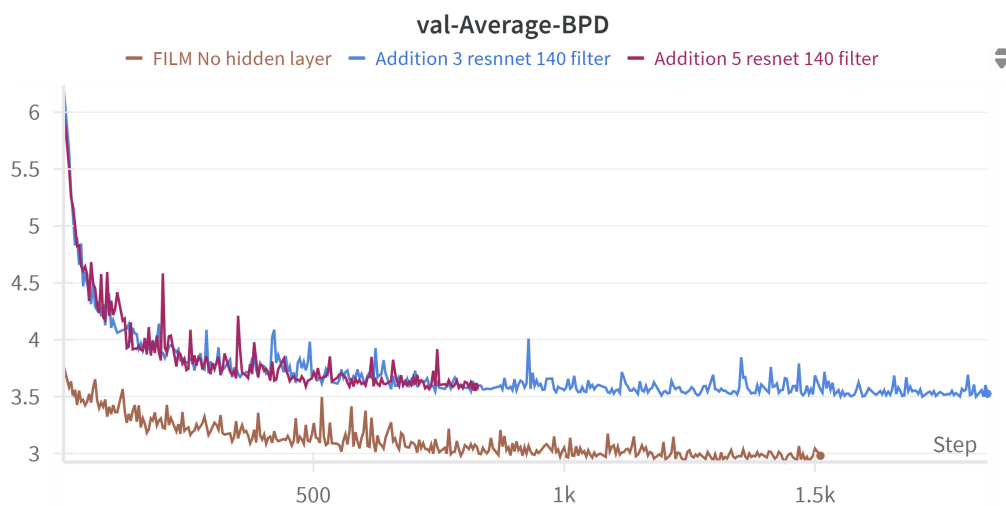


Figure 6: BPD comparison: FiLM vs. addition

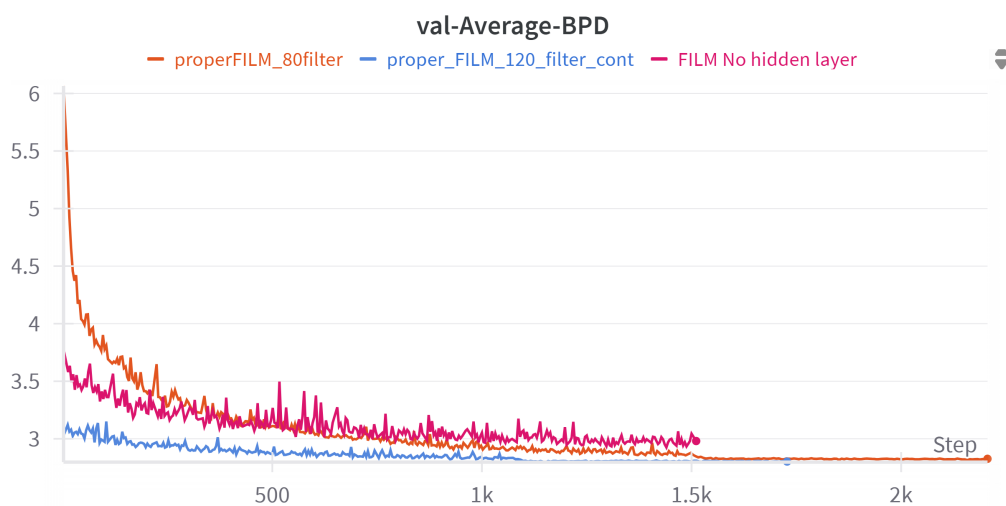


Figure 7: BPD across FiLM variants

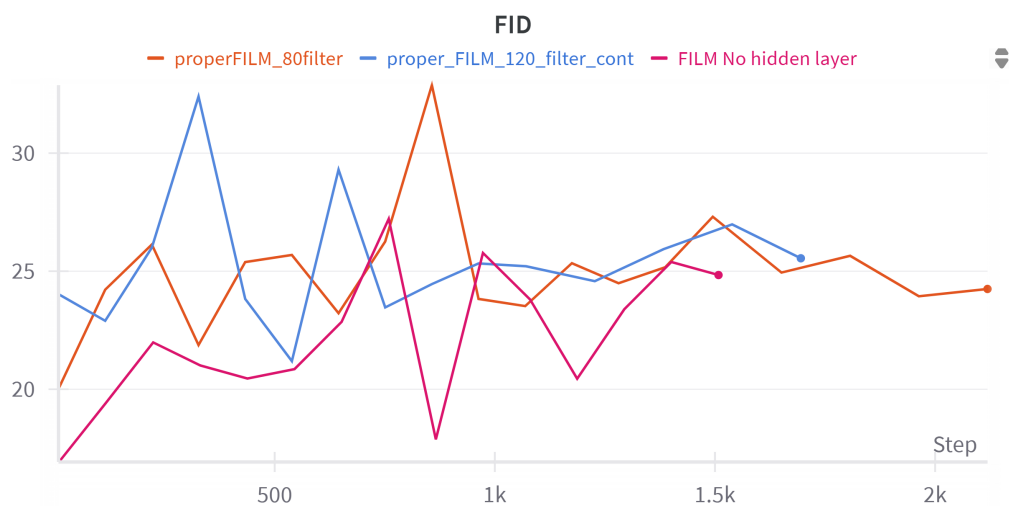


Figure 8: FID across FiLM variants

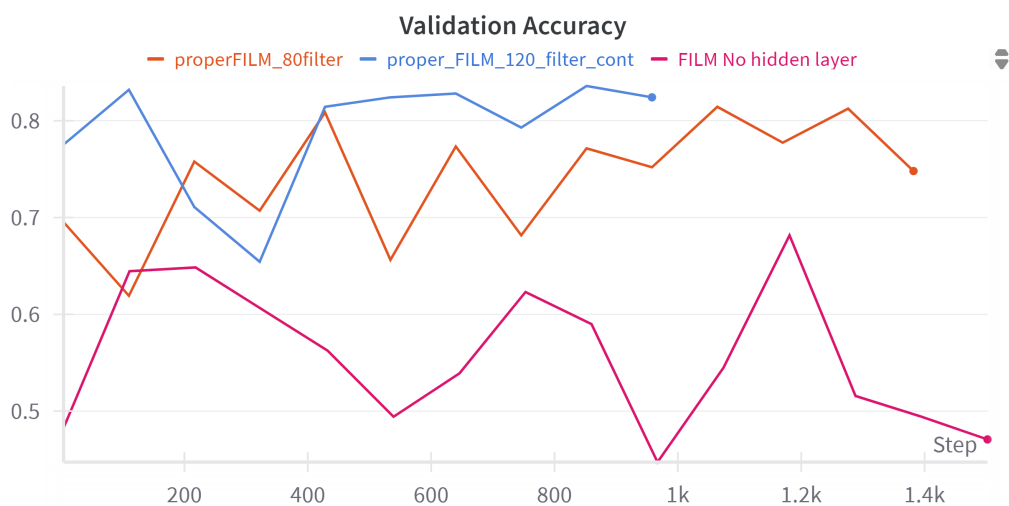


Figure 9: Validation accuracy across FiLM variants

References

- [1] T. Salimans et al., "PixelCNN++: Improving the PixelCNN with Discretized Logistic Mixture Likelihood," *arXiv:1701.05517*, 2017.
- [2] E. Perez et al., "FiLM: Visual Reasoning with a General Conditioning Layer," *arXiv:1709.07871*, 2017.
- [3] J. Hu et al., "Squeeze-and-Excitation Networks," *arXiv:1709.01507*, 2017.
- [4] P. Izmailov et al., "Averaging Weights Leads to Wider Optima and Better Generalization," *arXiv:1803.05407*, 2018.
- [5] A. van den Oord et al., "Pixel Recurrent Neural Networks," *arXiv:1601.06759*, 2016.
- [6] L. Chen et al., "PixelSNAIL: An Improved Autoregressive Image Model," *arXiv:1712.09763*, 2017.